

Operations on Data Structures

Q. Discuss the various fundamental operations that can be performed on data structures, explaining each with a suitable example.

> **Definition to Remember**

The Six Fundamental Operations

| Operation | Definition | Example |

Traversal Algorithm Example (Array)

```text

## Trade-offs Between Data Structures

| DS | Fast Operations | Slow Operations |

---

> **Must-Write Points (for 10 marks)**

---

> **Quick Recall**

> `Tra  
structu

---

## Algorithmic Implementations & Executable C Codes

### 1. Array Operations: Traversal, Insertion & Deletion (C Code)

```
```c
```

#includ

```
void traverse(int arr[], int n) {
```

printf("

```
int insertElement(int arr[], int n, int capacity, int element, int pos) {
```

if (n >=

```
int deleteElement(int arr[], int n, int pos) {
```

if (pos

```
int main() {
```

int arr[

```
printf("--- Initial Array ---\n");
```

travers

```
printf("\n--- Inserting 25 at Index 2 ---\n");
```

```
n = ins
```

```
printf("\n--- Deleting Element at Index 3 (30) ---\n");
```

```
n = de
```

```
return 0;
```

```
}
```

```
---
```

2. Searching Algorithms: Linear Search vs. Binary Search (C Code)

```
``c
```

```
#includ
```

```
// Linear Search - O(n)
```

```
int line
```

```
// Binary Search - O(log n) [Requires Sorted Array]
```

```
int bina
```

```
int main() {
```

```
int arr[
```

```
int linRes = linearSearch(arr, n, target);
```

```
printf("
```

```
int binRes = binarySearch(arr, n, target);
```

```
printf("
```

```
return 0;
```

```
}
```

```
---
```

3. Sorting & Merging Operations (C Code)

```
``c
#include <stdio.h>

// Bubble Sort Operation - O(n^2)
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(&arr[j], &arr[j + 1]);
}

// Merging Two Sorted Arrays - O(m + n)
void mergeArrays(int A[], int m, int B[], int n, int C[])
{
    int i = 0, j = 0, k = 0;
    while (i < m & j < n)
        if (A[i] < B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];
    while (i < m)
        C[k++] = A[i++];
    while (j < n)
        C[k++] = B[j++];
}

int main() {
    int A[] = {1, 3, 5, 7, 9};
    int B[] = {2, 4, 6, 8, 10};
    int C[10];

    mergeArrays(A, m, B, n, C);

    printf("Merged Array: ");
    for (int i = 0; i < 10; i++)
        printf("%d ", C[i]);
    printf("\n");

    return 0;
}
```